ECOLE NATIONALE DES SCIENCES APPLIQUEES

Beni Mellal – Filiere IACS

Pr. BE ELBAGHAZAOUI

COMPTE RENDU

TP N°2

Monolithe Modulaire

DTOs, Mappers et Service Layer – Spring Boot

Etudiant : Achraf HAMOUDA
Module : Architecture Logicielle
Date : 30 Mars 2026
Encadrant : Pr. BE ELBAGHAZAOUI

Annee Universitaire 2025–2026

Contents

1 Objectifs **2**

2 Architecture Modulaire **2**
2.1 Diagramme des dependances 2

3 Implementation **2**
3.1 DTOs 2
3.2 Mapper MapStruct 3
3.3 Service Layer 3
3.4 Controller adapte avec DTOs 4

4 Tests avec Postman **4**
4.1 Test 1 – Creer un produit (POST /api/products) 4
4.2 Test 2 – Lister les produits (GET /api/products) 5
4.3 Test 3 – Creer un client (POST /api/customers) 6
4.4 Test 4 – Lister les clients (GET /api/customers) 7
4.5 Recapitulatif 8

5 Exercice – Modules Customer et Order **8**
5.1 Exercice 1 – Module Customer complet 8
5.2 Exercice 1 – Module Order complet 9

6 Avantages du Pattern DTO + Mapper **12**

7 Conclusion **12**

1. Objectifs

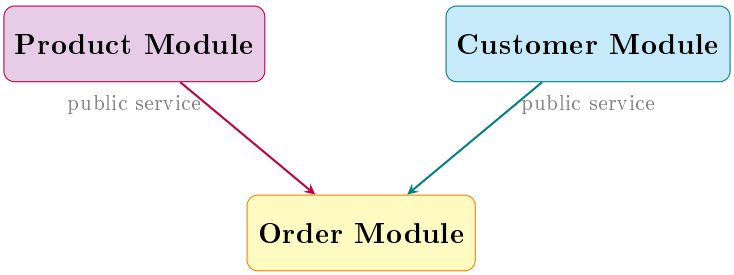
Ce TP vise a **restructurer le monolithe** du TP1 en modules metier independants, en appliquant les patterns DTO, Mapper et Service Layer pour preparer une future migration vers les microservices.

- Restructurer le monolithe en modules metier distincts (Product, Customer, Order)
- Definir des boundaries claires entre domaines
- Implementer les patterns **DTO**, **Mapper** (MapStruct) et **Service Layer**
- Implementer les modules Customer et Order complets
- Preparer le terrain pour une future migration microservices

2. Architecture Modulaire

Module	Composants	Dependances
Product	Controller / Service / Repository / DTO / Mapper	Aucune – module independant
Customer	Controller / Service / Repository / DTO / Mapper	Aucune – module independant
Order	Controller / Service / Repository / DTO / Mapper	ProductService + CustomerService

2.1 Diagramme des dependances



L'Order Module depend des **services publics** de Product et Customer – jamais directement de leurs repositories. Cela garantit un couplage faible et une separation claire des responsabilites.

3. Implementation

3.1 DTOs

Listing 1. ProductDTO.java

```java
@Data
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private Integer stock;
}
```

Listing 2. CreateProductRequest.java – Validation

```java
@Data
public class CreateProductRequest {
    @NotBlank(message = "Product name is required")
    private String name;
    private String description;
    @NotNull @Positive
    private BigDecimal price;
    @PositiveOrZero
    private Integer stock = 0;
}
```

3.2 Mapper MapStruct

Listing 3. ProductMapper.java – Mapping automatique

```java
@Mapper(componentModel = "spring")
public interface ProductMapper {
    ProductDTO toDTO(Product product);
    List<ProductDTO> toDTOList(List<Product> products);
    Product toEntity(CreateProductRequest request);

    @Mapping(target = "id", ignore = true)
    void updateEntity(CreateProductRequest request, @MappingTarget
        Product p);
}
```

3.3 Service Layer

Listing 4. ProductServiceImpl.java – Interface + Implementation

```java
@Service
@RequiredArgsConstructor
public class ProductServiceImpl implements ProductService {
    private final ProductRepository repository;
    private final ProductMapper mapper;

    public List<ProductDTO> getAllProducts() {
        return mapper.toDTOList(repository.findAll());
    }

    public ProductDTO createProduct(CreateProductRequest request) {
        Product product = mapper.toEntity(request);
        return mapper.toDTO(repository.save(product));
    }
```

```java

    public ProductDTO updateProduct(Long id, CreateProductRequest request) {
        Product product = repository.findById(id)
            .orElseThrow(() -> new RuntimeException("Not found: " + id));
        mapper.updateEntity(request, product);
        return mapper.toDTO(repository.save(product));
    }
}
```

3.4 Controller adapte avec DTOs

Listing 5. ProductController.java – Retourne des DTOs

```java
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;

    @GetMapping
    public ResponseEntity<List<ProductDTO>> getAll() {
        return ResponseEntity.ok(productService.getAllProducts());
    }

    @PostMapping
    public ResponseEntity<ProductDTO> create(
            @Valid @RequestBody CreateProductRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(productService.createProduct(request));
    }

    @PutMapping("/{id}")
    public ResponseEntity<ProductDTO> update(@PathVariable Long id,
            @Valid @RequestBody CreateProductRequest request) {
        return ResponseEntity.ok(productService.updateProduct(id, request));
    }
}
```

4. Tests avec Postman

4.1 Test 1 – Creer un produit (POST /api/products)

```
POST http://localhost:8080/api/products
Content-Type: application/json
{ "name": "Ipad", "price": 8000, "stock": 5 }
```

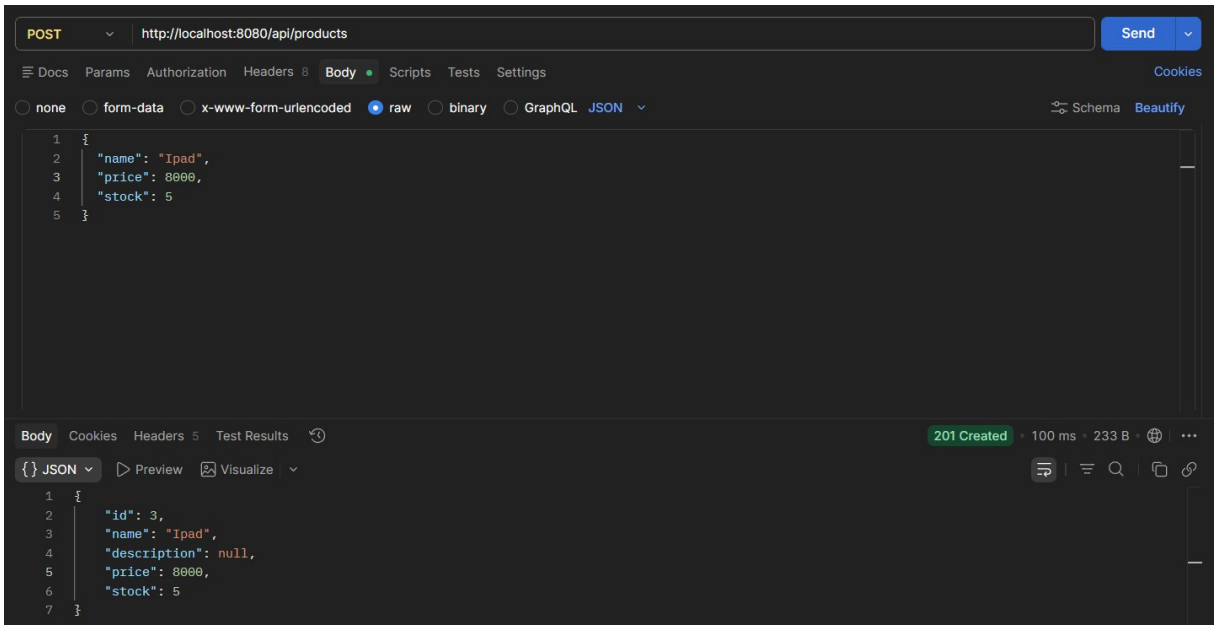


Figure 1. POST /api/products – 201 Created, ProductDTO retourne avec id=3

Resultat

`HTTP 201 Created` – ProductDTO retourne avec id=3. Mapping correct via `ProductMapperImpl`.

4.2 Test 2 – Lister les produits (GET /api/products)

```
GET http://localhost:8080/api/products
```

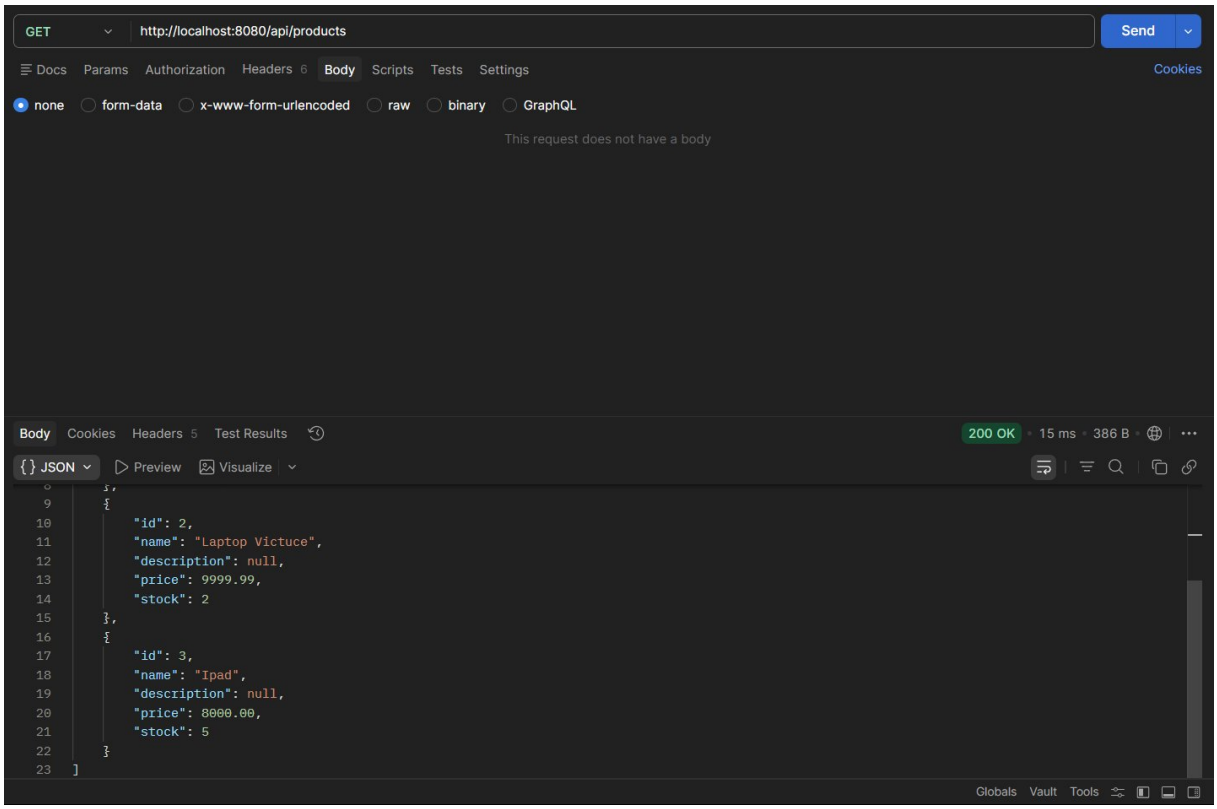


Figure 2. GET /api/products – 200 OK, liste de ProductDTOs

Resultat

`HTTP 200 OK` – Liste de ProductDTOs retournee (id, name, description, price, stock).

4.3 Test 3 – Creer un client (POST /api/customers)

```
POST http://localhost:8080/api/customers
Content-Type: application/json
{ "name": "Ahmed Benali", "email": "ahmed@gmail.com" }
```

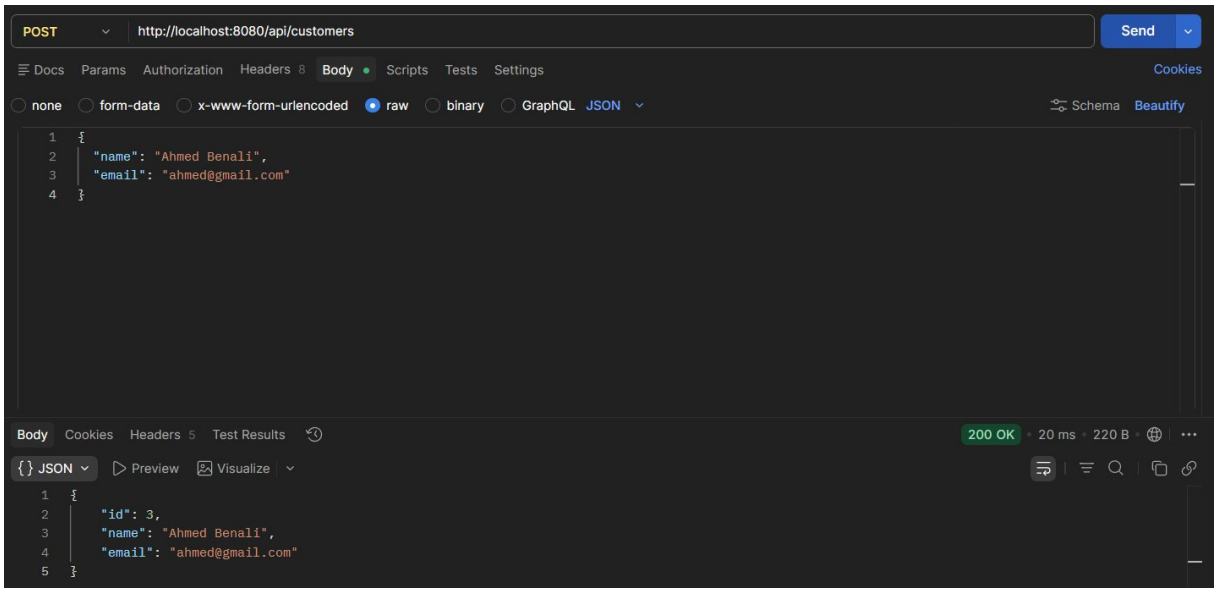


Figure 3. POST /api/customers – 200 OK, CustomerDTO retourne

Resultat

`HTTP 200 OK` – CustomerDTO retourne avec id=3, name et email correctement mappes.

4.4 Test 4 – Lister les clients (GET /api/customers)

```
GET http://localhost:8080/api/customers
```

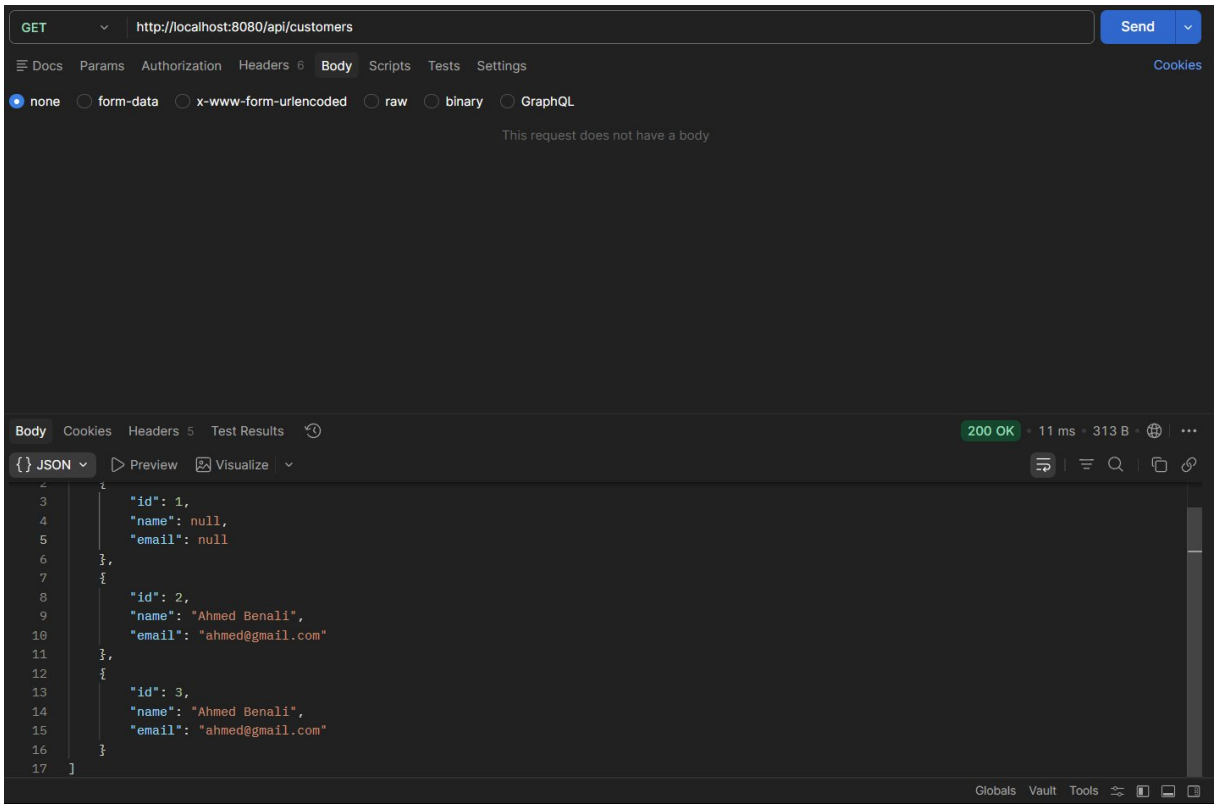


Figure 4. GET /api/customers – 200 OK, liste de CustomerDTOs

Resultat

`HTTP 200 OK` – Liste de CustomerDTOs retournee avec les donnees correctes.

4.5 Recapitulatif

Test	Methode	URL	Code
Creer produit	POST	`/api/products`	201
Lister produits	GET	`/api/products`	200
Creer client	POST	`/api/customers`	200
Lister clients	GET	`/api/customers`	200

5. Exercice – Modules Customer et Order

5.1 Exercice 1 – Module Customer complet

Listing 6. Customer.java – Entite

```
@Entity
@Table(name = "customers")
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank private String name;
    @Email private String email;
}
```

Listing 7. CustomerDTO.java et CreateCustomerRequest.java

```java
// DTO
@Data
public class CustomerDTO {
    private Long id;
    private String name;
    private String email;
}

// Request
@Data
public class CreateCustomerRequest {
    @NotBlank(message = "Name is required") private String name;
    @Email(message = "Invalid email") private String email;
}
```

Listing 8. CustomerMapper.java

```java
@Mapper(componentModel = "spring")
public interface CustomerMapper {
    CustomerDTO toDTO(Customer customer);
    List<CustomerDTO> toDTOList(List<Customer> customers);
    Customer toEntity(CreateCustomerRequest request);
}
```

Listing 9. CustomerServiceImpl.java – avec verification existence (Exercice 2)

```java
@Service
@RequiredArgsConstructor
public class CustomerServiceImpl implements CustomerService {
    private final CustomerRepository repository;
    private final CustomerMapper mapper;

    public List<CustomerDTO> getAllCustomers() {
        return mapper.toDTOList(repository.findAll());
    }

    public CustomerDTO createCustomer(CreateCustomerRequest request) {
        return mapper.toDTO(repository.save(mapper.toEntity(request)));
    }

    // Exercice 2 : methode qui verifie l'existence du Customer
    public boolean customerExists(Long id) {
        return repository.existsById(id);
    }

    public Customer getCustomerEntityById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Customer not found with id: " + id));
    }
}
```

5.2 Exercice 1 – Module Order complet

Listing 10. Order.java – Entite avec relations

```java
@Entity
@Table(name = "orders")
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "customer_id", nullable = false)
    private Customer customer;

    @ManyToOne
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    private Integer quantity;
    private BigDecimal totalPrice;

    @Column(name = "order_date")
    private LocalDateTime orderDate = LocalDateTime.now();

    private String status = "EN_ATTENTE";
}
```

Listing 11. OrderDTO.java et CreateOrderRequest.java

```java
@Data
public class OrderDTO {
    private Long id;
    private Long customerId;
    private String customerName;
    private Long productId;
    private String productName;
    private Integer quantity;
    private BigDecimal totalPrice;
    private String status;
    private LocalDateTime orderDate;
}

@Data
public class CreateOrderRequest {
    @NotNull private Long customerId;
    @NotNull private Long productId;
    @NotNull @Positive private Integer quantity;
}
```

Listing 12. OrderServiceImpl.java – depend de ProductService + CustomerService

```java
@Service
@RequiredArgsConstructor
public class OrderServiceImpl implements OrderService {
    private final OrderRepository repository;
    private final OrderMapper mapper;
    private final CustomerService customerService; // service public,
        pas repository
    private final ProductService productService;   // service public,
        pas repository
```

```java

    public OrderDTO createOrder(CreateOrderRequest request) {
        // Verification existence Customer (Exercice 2)
        if (!customerService.customerExists(request.getCustomerId()))
            {
            throw new RuntimeException("Customer not found: "
                + request.getCustomerId());
        }
        Customer customer = customerService
            .getCustomerEntityById(request.getCustomerId());
        Product product = productService
            .getProductEntityById(request.getProductId());

        Order order = Order.builder()
            .customer(customer)
            .product(product)
            .quantity(request.getQuantity())
            .totalPrice(product.getPrice()
                .multiply(BigDecimal.valueOf(request.getQuantity())))
            .build();
        return mapper.toDTO(repository.save(order));
    }

    // Exercice 3 : historique des commandes d'un client
    public List<OrderDTO> getOrdersByCustomer(Long customerId) {
        if (!customerService.customerExists(customerId)) {
            throw new RuntimeException("Customer not found: " +
                customerId);
        }
        return mapper.toDTOList(
            repository.findByCustomerIdOrderByOrderDateDesc(
                customerId));
    }
}
```

Listing 13. OrderRepository.java – requete custom pour historique

```java
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {
    // Exercice 3 : requete pour l'historique d'un client
    List<Order> findByCustomerIdOrderByOrderDateDesc(Long customerId)
        ;
}
```

Listing 14. OrderController.java – endpoint historique (Exercice 3)

```java
@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {
    private final OrderService orderService;

    @PostMapping
    public ResponseEntity<OrderDTO> create(
            @Valid @RequestBody CreateOrderRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(orderService.createOrder(request));
    }
```

```java

    @GetMapping
    public ResponseEntity<List<OrderDTO>> getAll() {
        return ResponseEntity.ok(orderService.getAllOrders());
    }

    // Exercice 3 : historique des commandes d'un client
    @GetMapping("/customer/{customerId}")
    public ResponseEntity<List<OrderDTO>> getByCustomer(
            @PathVariable Long customerId) {
        return ResponseEntity.ok(
            orderService.getOrdersByCustomer(customerId));
    }
}
```

Endpoint historique : `GET /api/orders/customer/{customerId}` retourne toutes les commandes d'un client triees par date decroissante. Si le client n'existe pas, une erreur 404 est retournee grace a la verification `customerExists()`.

6. Avantages du Pattern DTO + Mapper

- **Separation des couches** : l'entite JPA ne sort jamais de la couche service, evitant l'exposition de details internes.
- **Flexibilite** : le DTO peut exposer uniquement les champs necessaires, sans modifier l'entite.
- **Validation propre** : le `CreateProductRequest` valide les entrees independamment du modele de base.
- **MapStruct** : generation automatique du code de mapping a la compilation, sans reflection et sans overhead runtime.
- **Preparation microservices** : chaque module expose une interface de service publique – facilite la decoupage futur.

7. Conclusion

Ce TP a permis de refactoriser le monolithe du TP1 en une architecture modulaire propre. L'introduction des patterns **DTO**, **Mapper** (MapStruct) et **Service Layer** ameliore significativement la maintenabilite et prepare la base de code pour une migration vers les microservices (TP4). Les trois modules (Product, Customer, Order) sont maintenant independants avec des interfaces de communication bien definies.